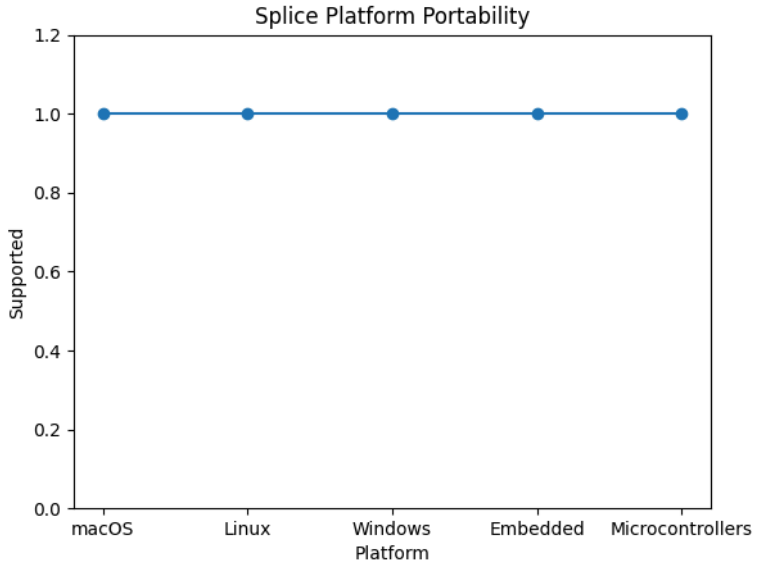
Core Benefits of Splice

Github: https://github.com/Open-Splice/Splice/

1. Write Once, Run Anywhere (Real Portability)

- Same Splice code runs on **macOS, Linux, Windows, embedded systems, and microcontrollers**
- Platform differences handled by the **Splice VM**, not the programmer

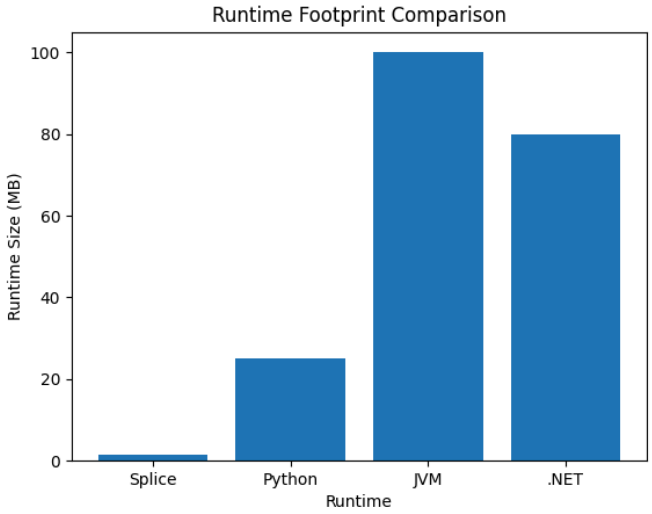


---

2. Small Runtime Footprint

- Designed to run in **low-memory environments**
- Suitable for **embedded devices, firmware, IoT, and custom OS kernels**

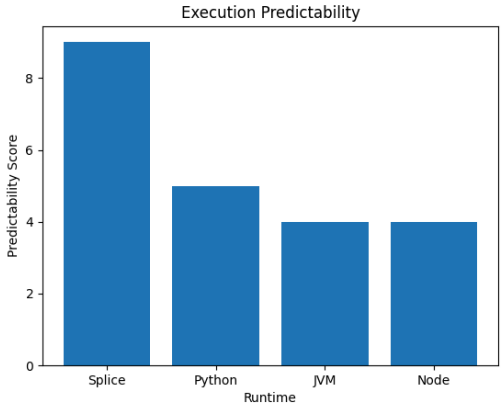
- Much smaller than JVM, .NET, or Python runtimes



---

3. Predictable Execution

- No hidden background threads
- No heavy garbage collector pauses
- Execution flow is explicit and deterministic
  → important for **systems, embedded, and real-time-ish workloads**



---

4. C-Level Integration

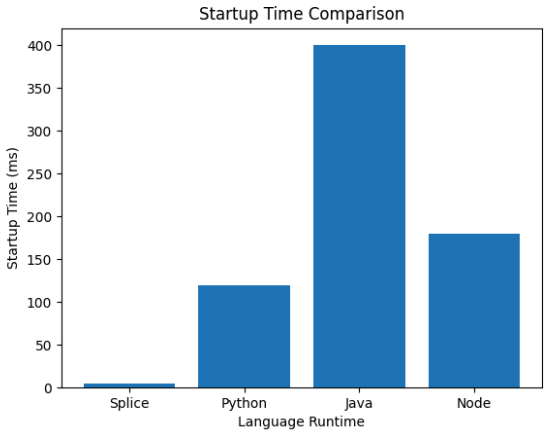
- Written in C and designed to embed easily into:
    - Existing C/C++ projects
    - Firmware
    - Game engines
    - Operating systems

Native functions can be exposed directly to Splice

---

5. Fast Startup

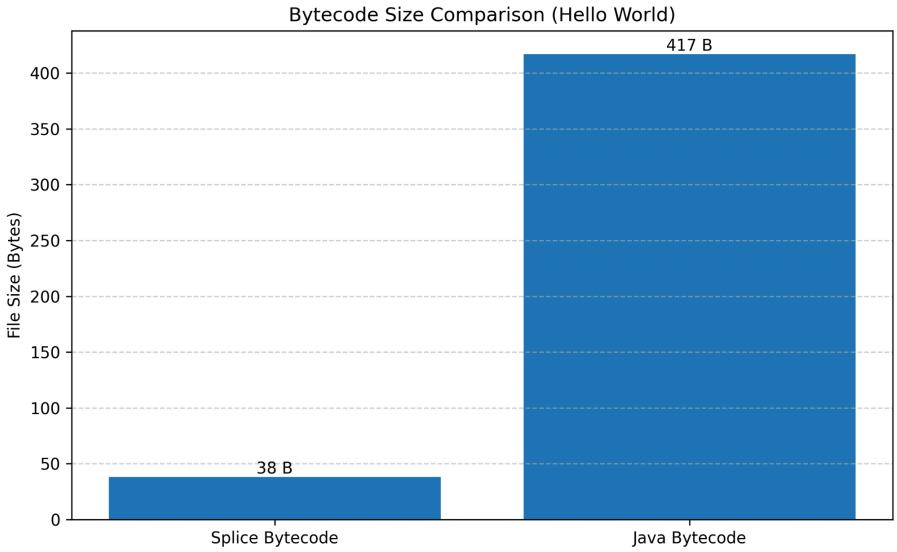
- No VM warm-up
- No JIT delay
- Programs start immediately
  → ideal for **CLI tools, scripts, system utilities**



---

6. Custom Bytecode (SPBC / Groupcode)

- Bytecode is:
  - Compact
  - Readable (ASCII-grouped)
  - Easy to debug and inspect
- Enables tooling like:
  - Hex inspection
  - Static analysis
  - Custom loaders

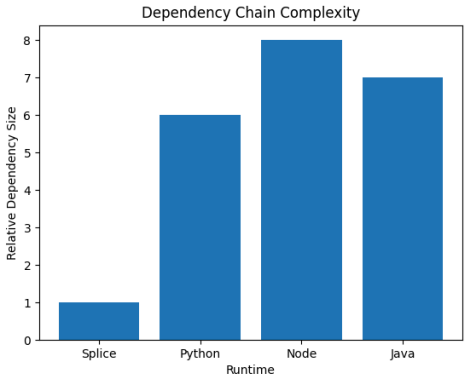


---

7. Minimal Dependency Chain

- No massive standard library requirement

- No external runtime downloads
- Works without internet or package managers
  → good for **offline, secure, or controlled environments**



---

8. Explicit Language Design

- Simple syntax
- No hidden magic
- Clear control flow
- Easy to reason about generated bytecode

---

9. Embeddable VM Architecture

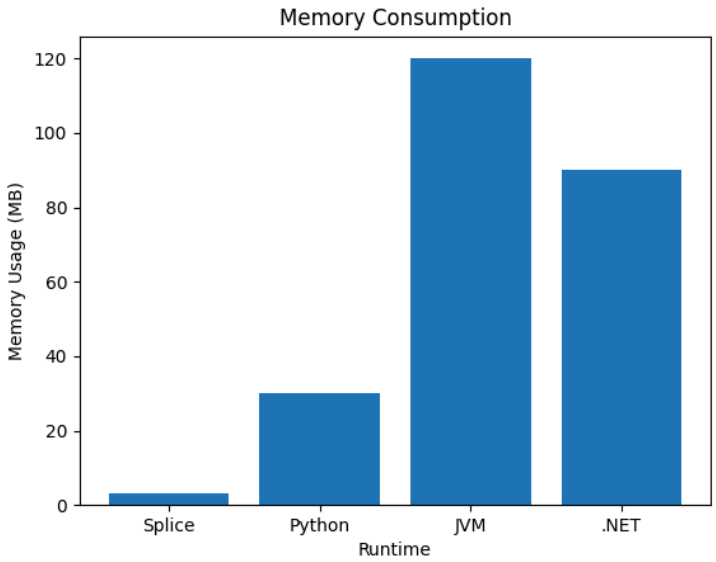
- Splice VM can act as:
  - A scripting engine
  - Firmware logic layer

- Plugin system
- Command processor

---

10. Customizable Memory Model

- Can replace or wrap `malloc`



- Supports arena allocators / custom allocators
- Useful for memory-constrained or performance-critical systems

---

11. Stable Cross-Platform Behavior

- Same semantics across platforms
- No “works on my machine” runtime differences

- Ideal for CI/CD and reproducible builds

---

12. Language + Toolchain Control

- You control:
    - Syntax
    - Bytecode
    - VM behavior
    - Build system
- No vendor lock-in
- No dependency on big ecosystems

---

13. Good Fit for System-Adjacent Software

Splice works especially well for:

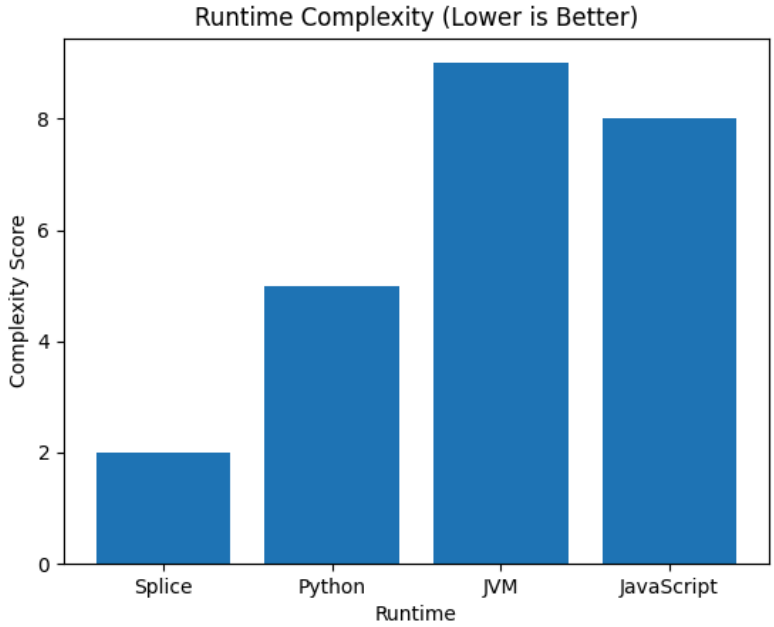
- Firmware scripting
- Game engines
- Build tools
- Custom OS utilities
- Embedded AI logic

- Developer tools

---

14. Low Cognitive Overhead

- Easier to understand end-to-end than:
    - JVM stack
    - CPython internals
    - JavaScript engines
- You can understand **lexer** → **parser** → **bytecode** → **VM** fully



---

15. Long-Term Maintainability

- No rapid breaking changes from external vendors

- Backward compatibility can be enforced at the VM level
- Ideal for long-lived systems and products like Banking systems, and legacy systems that can be hard to give updates